EUROGRAPHICS'98 / N. Ferreira and M. Göbel
(Guest Editors)

Volume 17, (1998), Number 3

Maximum Intensity Projection Using Splatting in Sheared Object Space

Wenli Cai and Georgios Sakas

Fraunhofer Institute for Computer Graphics, Rundeturmstrasse 6, 64283 Darmstadt, Germany
email: {wcai, gsakas}@igd.fhg.de

Abstract

In this paper we present a new Maximum Intensity Projection (MIP) algorithm which was implemented employing splatting in a shear-warp context. This algorithm renders a MIP image by first splatting each voxel on two intermediate spaces called "worksheet" and "shear image". Then, the maximum value is evaluated between worksheet and shear image. Finally, shear image is warped on the screen to generate the result image. Different footprints implementing different quality modes are discussed. In addition, we introduced a line encoded indexing speed-up method to obtain interactive speed. This algorithm allows for a quantitative, predictable trade-off between interactivity and image quality.

Keywords: Volume rendering, MIP, splatting, shear warp

1. Introduction

Volume rendering is well-known as an accumulation process, in which each pixel intensity is calculated by accumulated the voxel intensities and opacities along the line of view [DrCH88, Kaji84, Levo88]. Volume rendering algorithms are divided in two fundamental types: image space and object space methods. Ray casting [Levo88] is the typical algorithm of image space volume rendering. Object space methods include V-buffer [UpKe88], splatting [West90], and shear-warp factorisation [LaLe94]. Object space volume rendering is superior to ray casting for four main reasons: gapless access of the data space, optimal caching, highly incremental behaviour, and several speed-up methods allowing data set to be preprocessed before projection to avoid unnecessary voxel traversal, like Hierarchical Splatting [LaHa91], Runlength coding [LaLe94], Block Compression [Knit95] and Indexing [IhLe95]. Compression in frequency domain [LiGK97] is another object space rendering techniques. This results in general in a much higher speed of object space methods coupled with a high degree of quality.

Usually, each voxel is regarded as a semi-transparent material. The result of such accumulations can be presented as (semi-transparent) surface or cloud. Another technique of volume rendering called Maximum Intensity Projection (MIP) is mainly applied in MRA and, more recently, 3D-ultrasonic data sets. Its principle is surprisingly simple: each voxel is regarded as a self-emitting cubical light source and the final pixel intensity is computed as the maximum voxel value among all voxels projected on the pixel along the viewing direction. Thus, in MIP only projection and comparison between voxel values is requested, accumulation and opacity blending are not employed. Due to the difference between maximum comparison and accumulation, most MIP algorithms use only ray casting to traverse volume data and select the maximum value among all accessed voxels along a ray. Unfortunately, the well-known volume rendering acceleration techniques (early ray termination, pyramid structure like octree) do not directly apply on MIP since most of them have been developed for accumulation volume rendering. Currently, one usually has to traverse all voxels on a ray in order to select the maximum value among them. Therefore MIP ray casting is time consuming and far from interactive rendering speed. The main difficulty is how to evaluate the maximum value of a pixel in object space by voxel order traversing. The scope of this paper is to explore a fast MIP algorithm in object space.

[HeMS95] described a Z-buffer assisted MIP method. After object order traversing and projection, the Z-component of each point is set to its intensity, thus allow-

ing to utilise the Z-buffer hardware to accelerate rendering. Another way to accelerate MIP rendering was proposed in [ZuKV94]: MIP is speeded up by Distance Coding of data set for ray casting traversal. A high quality maximum evaluation method was reported in [SaGS96], which is also based on ray casting.

The shear-warp factorisation rendering algorithm [LaLe94] belongs to the fastest object space rendering algorithms. The advantages of shear-warp factorisation are that sheared voxels are viewed and projected only along ±X, ±Y and ±Z axis, i.e. along the principle viewing directions in sheared object space rather than an arbitrarily viewing direction, which makes it possible to traverse and evaluate the maximum value slice by slice. Neighbour voxels share the same footprint and are exactly one pixel apart. A shear-warp MIP implementation for thin slab is reported in [YeNR96].

We chose shear-warp algorithm as the volume rendering context for our MIP rendering due to its superior speed and algorithmic advantages. This paper proposes a MIP method using splatting in sheared object space, which is both more accurate and faster than the original shear-warp proposed in [LaLe94] and [YeNR96]. The paper focuses on different aspects of the algorithm:

- Section 2 discusses principal difficulties in the classical MIP splatting and proposes our solution for shearing voxels in addition to slices, how to perform MAX comparisons, as well as dealing with intra- and inter-slice interpolation alternatives
- Section 3 discriminates three different footprints for three different MIP quality modes
- Section 4 presents a novel line encoded indexing technique used to accelerate MIP
- Finally, in section 5 we present experimental results (runtimes) and discussion

2. Splatting in Sheared Object Space

In shear-warp the complete transformation matrix is decomposed into two transformations, a shear matrix and a warp matrix. The dataset is sheared first according to the shear transformation and an intermediate image, which we called “shear image”, is generated by voxel projection in sheared space. Each voxel is splatted according to its footprint in sheared object space. After all voxels have been splatted, the intermediate shear image is warped to create the result image displayed on the screen according to the warp transformation. In this paper, we only consider orthographic (parallel) projection.

2.1 Problems in MIP Splatting

Although splatting is a straightforward method for projection, it is difficult to be applied in MIP since our goal is to evaluate the maximum value at each one pixel rather than to accumulate voxel contributions. Usually one pixel receives contributions from several voxels. As shown in Figure 1, the intensity of pixel (a) should obviously be 250 since the pixel is covered by two grey voxels each one with the value 250. In the original splatting algorithm each one of the two voxels will contribute half of its intensity to the pixel. Thus, if we compare each voxel contribution individually with the pixel value to evaluate the maximum (MAX operation), the pixel intensity would be 125 rather than 250! A possible solution here could be to assign the maximum value of each voxel without calculating the voxel contribution first. This “solution” will, however, create strong alias in the form of stair-case edges and will thicken thin lines. In addition, the second example shows that this solution will also fail calculating the correct intensity value: Pixel (b) receives contributions from two different voxels, one having the value 250 and one the value 150. Due to the fact that voxel values are interpolated in object space and assuming for simplicity a linear interpolation scheme, the correct value of the pixel (b) should be (250 + 150) / 2 = 200. Instead, by calculating the voxel contribution first and comparing each value individually the pixel would obtain the value 250/2 = 125! Alternatively, if contributions are not interpolated, the pixel value will be set to 250! In both cases the correct value of 200 will be missed.

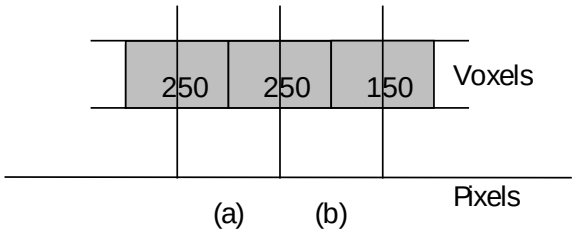


Figure 1. The Situation in MIP Splatting

In the original shear-warp algorithm [LaLe94] only slices are sheared rather than each voxel itself (see the dotted line in Figure 2 left). In other words, slices are displaced (sheared) relatively to each other, but the voxels within each slice are remaining orthogonal cubes. In the MIP implementation proposed in [YeNR96], the voxel sampling value is evaluated along the line emitting from the pixel centre, a kind of point sampling. We call this case “projective shear-warp” in order to discriminate it from “splatting shear-warp” to be introduced by us in this paper. In projective shear-warp, the voxel projection is obviously not accurately calculated. The projection area of one voxel is exactly equal to one. In splatting shear-warp we instead calculate the mathematically correct shear of the object space. The complete dataset is regarded to be a continuous space sheared by the shearing transformation. Thus, each voxel is sheared as well, resulting in parallelepipeds rather than in cubes. Therefore the projection area of a voxel is now in general greater than 1.0. This differ-

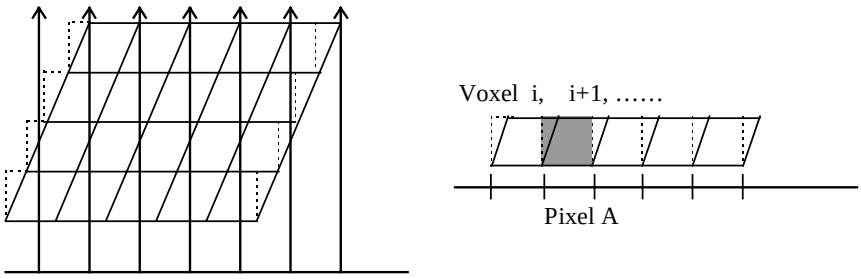


Figure 2 Difference between Voxel Splatting and Projection in shear-warp

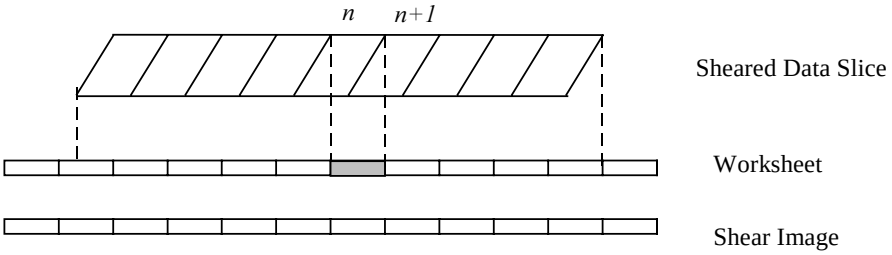


Figure 3. Worksheet and Shear Image

ence is illustrated in Figure 2 right: in projective shear-warp pixel A only samples the value of voxel *i+1* (dashed line). In splatting shear-warp instead, pixel A samples values from both voxel *i* and *i+1* (solid line). The implication of this is that in the later case a sub-voxel splatting sampling calculation according to an individual splatting table (footprint) becomes necessary.

2.2 Solution to the MIP Splatting Problem

- **The "Worksheet"**

In order to solve the first problem and calculate different voxel contributions to one pixel resulting from splatting parallelepipeds rather than cubes, we introduce an additional intermediate image plane called "worksheet". The worksheet is acting as an image buffer lying at the same location and has the same size as the shear image. For each sheared data slice, voxels are first splatted to the worksheet and values are accumulated in the usual way. After accumulating all contributions of one slice to the worksheet, pixels in the worksheet are compared with pixels in the shear image one by one to evaluate their maximum value, i.e. a MAX operation is performed between worksheet and shear image see Figure 3. The effect of introducing the worksheet is that a partial image is calculated first containing the contributions of one voxel-slice only before it is compared with the "final" shear image.

- **Standard Interpolation Mode**

We regard the *n*th voxel as a cube of constant intensity I_n and spatial location co-ordinates in [*n, n+1*]. In sheared object space, slices are parallel to each other, therefore the pixel intensities in the worksheet receive voxel contributions (interpolation between sheared voxels) only from one slice at the time. In this case interpolations are performed only in horizontal and vertical directions, and no inter slice contribution (depth direction interpolation) is considered (we will discuss the inter slice contribution later). In Figure 3, the dashed pixel in the worksheet accepts only the contributions from voxel *n* and *n+1*. Since the pixel size is the same as the voxel size, which is indicated by the orthographic splatting, one pixel accepts 2 voxels' contributions in 2D space (1D "image" generation like the case shown in Fig. 2 and 3), and 4 voxels in 3D space (2D image generation). This interpolation of voxel values only within one slice is called standard quality mode splatting in this paper.

- **Tri-Linear Interpolation Mode**

Concerning 3D linear interpolation or other splatting kernels, we have to consider the contributions from neighbour voxels, located in neighbouring slices to the worksheet, which we call inter-slice contribution. Figure 4 shows the situation: original voxel boundaries are shown as solid lines, voxel co-ordinates vary in the range from [*i-0.5, j-0.5*] to [*i+0.5, j+0.5*]. Due to tri-linear interpolation, interpolation regions are formed between the mid-points of neighbouring voxels forming the "cell interpolation" grid shown with dashed lines. Interpolation cell co-ordinates vary between [*i, j*] and [*i+1, j+1*]. The central voxel located at position (*i, j*) and shown in dark texture provides interpolation contributions to all co-ordinates in the range [*i-1, j-1*] and [*i+1, j+1*], i.e. it spans a 2×2×2 voxels area in 3D-linear interpolation space.

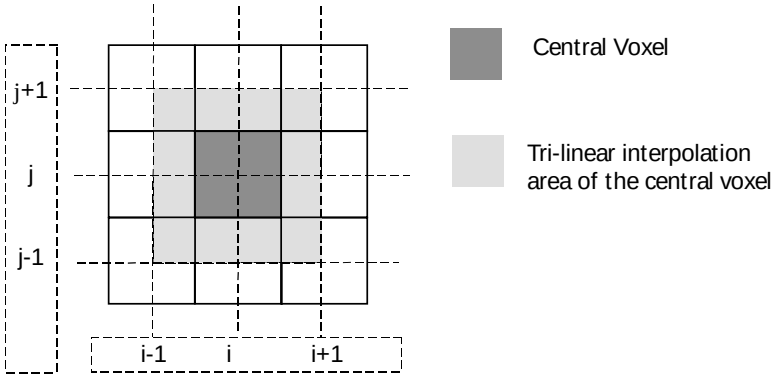


Figure 4. The areas influenced by the central voxel (i, j) during a tri-linear Interpolation

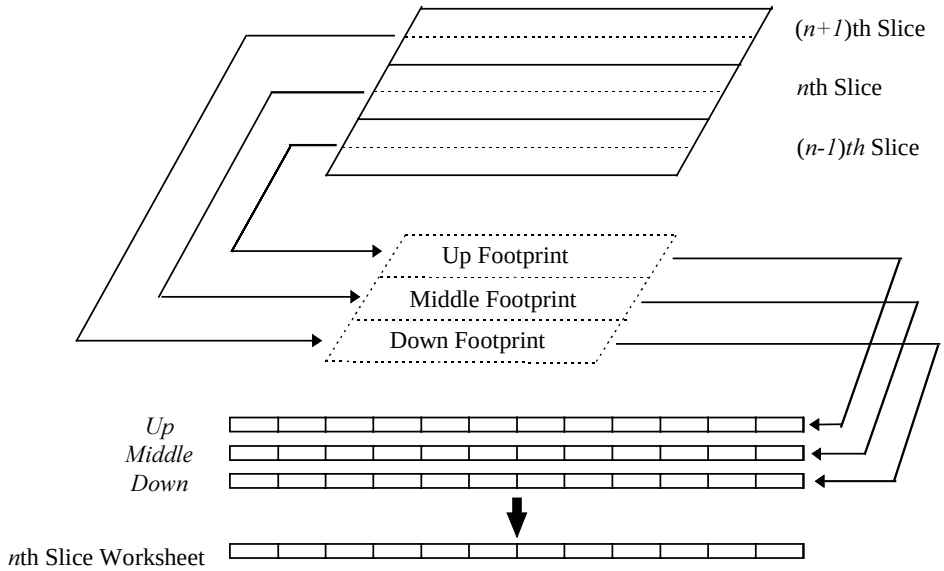


Figure 5. The Accumulation of Worksheet in 3D Linear Interpolation in 2D Case

However, since we splat the volume slice by slice, the contributions of the central voxel (*i, j*) to a corresponding pixel in the worksheet have to be considered among 3 slices, that is the *j*th slice, the (*j-1*)th slice and the (*j+1*)th slice (if *j* is the current splatting slice). These 3 different contributions are calculated with three different footprints (= weighting factors) called the "middle footprint" of slice *j*, "up footprint" of slice *j-1* and "down footprint" of slice *j+1*, see Figure 5. We will discuss the calculation of middle footprint, up footprint and down footprint in the next section.

During splatting, each voxel in these 3 slices is weighted by the up, middle or down footprint and splatted in one of three corresponding different buffers called up worksheet, middle worksheet and down worksheet, instead of direct projection to the current *n*th worksheet. When all voxels of the current slice are splatted, the contributions of down worksheet in the (*n+1*)th slice, middle worksheet in the *n*th slice, and up worksheet in the (*n-1*)th slice are summed up in the current worksheet. Last, the current worksheet is compared with the shear image to evaluate the MIP for the pixel. This also means that we have to maintain 3 worksheets plus the current *n*th worksheet[1]. In

[1] Note: for implementation speed reasons in order to optimize cache accesses it is desirable to avoid accessing the volume voxels several times. Therefore we choose to maintain 9 buffers instead of 3: each voxel is accessed only once and its up, middle and down contributions are calculated and stored in the corresponding buffers. Thus, we trade-of speed (optimal cache access) with additional memory. A typical buffer size is - depending on

addition, each voxel will perform 3 convolutions each one of the size of 5×5, compared with 1 convolution of the size of 3×3 in the previous standard quality mode. Therefore the computation cost is increased about 4~5 times. This mode is called high quality mode splatting.

3. Footprints for Different Quality Modes

3.1 Preview Mode Footprint

The first possible footprint is designed by the nearest neighbour point sampling method. If a pixel's centre is covered by a voxel's footprint, the voxel intensity is assigned to that pixel. This is the simplest way to evaluate the maximum value and it can be performed without worksheet. We called preview quality mode splatting in this paper. In the next section, we will discuss in detail its accelerating implementation with indexing and sorting. Obviously, this is the fastest mode but due to aliasing effects it used only for preview purposes.

3.2 Standard Mode Footprint

The footprint of a voxel is calculated according to its projection area in the sheared object space. Since the maximum shear displacement between neighbour slices is 1.0 (corresponding to a 45° rotation), the footprint is a 2×2 matrix (see Figure 6 left). Considering that the starting point of the footprint will vary between slices, the final convolution matrix is in general a 3×3 matrix, each element contains a weight, which is the voxel contribution to the corresponding pixel (see Figure 6 right). This matrix is the same for all voxels within one slice. In addition, since slices themselves are parallel to each other, also voxels located in different slices share the same general footprint shape with only a different translation offset. Therefore we can build a general 2×2 footprint table and only adapt it between slices: The 3×3 convolution matrix for each slice can be converted from this general 2 x 2 footprint table.

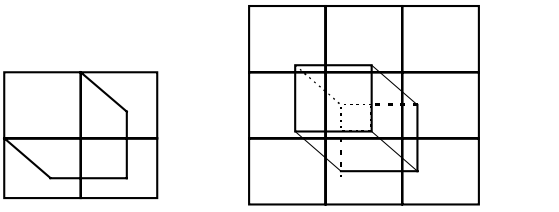


Figure 6. Shear Footprint and its Convolution Matrix

We establish a general footprint table by digitising and scanning the 2×2 shear footprint area. The size of the table is 2N×2N, where N×N is the digitisation degree i.e. the number of subpixels within one pixel, which is usually selected between 10 to 20 or even more, depending on the required accuracy.

While scanning the voxel in sheared object space, we compute at the location of the mid-point of each subpixel the "thickness", i.e. its projection length along the principle viewing direction (in Figure 7 the Z axis). In the standard mode, the weight of each subpixel is calculated by

$$W_i = \frac{l_i}{N * N}$$

where $l_i = |Z_1 - Z_0|$ and $N*N$ is the number of subpixels within one pixel.

For the complete general table, the weight summation is 1.0, i.e. it is normalised.

3.3 High Quality Mode Footprint

For the tri-linear interpolation kernel or other reconstruction kernels, the general table size is 4×4 since each voxel contributes to an interpolation cell with the size 2×2 as shown on Fig. 4. Unlike in the "standard" case, where the value of a pixel is regarded to be constant and therefore each weight is approximated simply by the voxel thickness at the mid-point of a subpixel, in the high quality case the "cell value" is changing along the z axis as a result of the tri-linear interpolation between neighbouring voxel values and therefore we calculate the weight by employing an integral. For each sub-pixel of a 4×4 sheared voxel, the weight is calculated by,

$$W_i = \int_{Z_0}^{Z_1} h_v(x_0, y_0, z)dz ,$$

where (x_0, y_0) is the centre of subpixel, $h_v(x, y, z)$ is the volume reconstruction kernel (in our case tri-linear interpolation), and (Z_0, Z_1) is the integral range. The distance between start and end point is divided into three integral ranges at ±0.5, for middle footprint, up footprint, and down footprint, respectively. Thus, three footprints are computed according to three different integral ranges.

In addition we assume that in the general case the voxel intensity is decaying to zero at the kernel boundary like this is the case in the tri-linear interpolation. Two symmetrical reconstruction kernels with boundary equal to zero have been implemented: the tri-linear interpolation kernel and a cosine bell filter kernel. For general interpolation schemes, footprint can be calculated more elegantly using the Fourier approach.

$$h_{linear} = 1 - |x|$$
$$h_{\cos} = 1 + \cos(\pi x / x_m) , \; x_m \text{ is the filter radius}$$

The volume reconstruction kernel can be written

the data size - 512 x 512 = 1/4th Mbyte or even less, therefore the additional memory requirements for the extra 6 buffers are ca 1½ Mbyte.

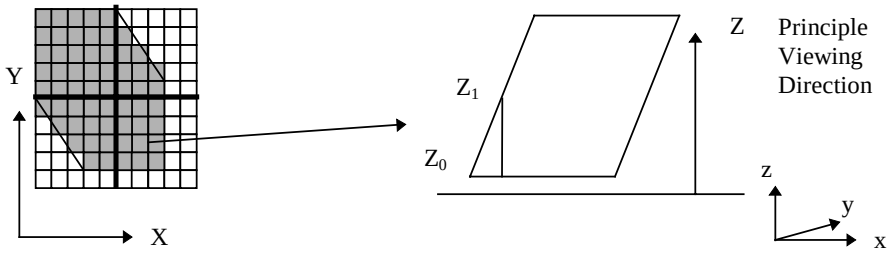


Figure 7. General table calculation

$h_v = h_s(x) * h_s(y) * h_s(z)$,

h_s is the kernel along the corresponding axis.

After choosing the reconstruction kernel, the general footprint table is calculated and is converted to a 3×3 (standard quality mode) or a 5×5 (high quality mode) convolution matrix for each slice according to its starting point. In order to avoid the time consuming convolution, we built a specific footprint table size of 3×3 or 5×5 according to different starting points for each slice. The value of the matrix element is the summed weights of all subpixels covered by this matrix element corresponding pixel. Also, the offset to element (0, 0) of each element in the matrix is directly stored in the table and the weight is converted from one double type to a table size of 256 unsigned long by *weight*grey* (grey range from 0 -255) to avoid the double type multiplication during rendering. The data structure of the footprint matrix is,

```
struct {short count;   /* number of hit elements by the
                          general footprint table */
        struct node  { long offset;  /* offset between cov-
                          ered element and element (0,0) */
                       unsigned long weight [256];
}}
```

With this data structure, the convolution is converted to the following code;

```
for (i=0; i<count; i++)
        *(sheet_point + offset)=weight [grey];
```

4. Accelerating Splatting by Line Encoded Indexing

In order to accelerate splatting especially for a fast preview of volume data, we introduced a line encoded indexing method. The data set is still processed slice by slice. Within each slice, voxels chains are run-length coded within each line. A chain contains the grey value and the length. If the splatting mode is "preview" (= nearest neighbour, no kernel weighting), an additional sorting of the chains is employed: After all lines of a slice have been run-length encoded, all chain segments for the slice are then sorted by intensity from high to low, i.e. from 255 to 0. Each chain contains now the grey value (implicitly, since all chains of a given grey are stored within the same "container"), the run-length and the chain's (x, y) position within the slice (see Figure 8 for the encoding data structure). Sorting of the values from high to low has the target to accelerate the MIP process in the preview mode, since chains with large values will be splatted first. In this case a comparison between the actual pixel value and the splatted value is not necessary: since we splat from large to small values, if a pixel already has a value assigned, it has not to be regarded again![2]

For each data set, chains are created for all 3 principle directions, i.e. x, y and z. Depending on the dataset orientation and, thus, the principle splatting direction, the corresponding chain encoding set is employed. However, for a given dataset, chains (and in the preview mode, their sorting) are created only once, removing the on-line computation during rendering to a pre-processing stage.

When standard or high quality kernels are used, sorting as above is not possible due to the inter-line contribution and the inter-slice contribution. However, building run-length chains is still beneficial due to the two acceleration effects: first, since we splat lines instead of voxels, the convolution per voxel is changed to convolution per line segment. Due to the fact that all voxels within a chain have the same value, convolution with the full kernel is necessary for the first and the last chain voxel only, intermediate voxels are splatted using a simpler kernel. With increasing line length, the computation needed for convolution is becoming increasingly smaller. The second acceleration effect comes from the fact that the number of memory accesses is reduced by the average chain length. This later acceleration effect is true also for the preview splatting mode using sorted chains.

[2] The sorting of chains according to their gray value can be performed for the complete volume as well. We decided to sort within each slice rather than for the whole volume in order to avoid large sorting procedures as well as for saving the memory for storing the z co-ordinate of a chain. However, if memory considerations and higher pre-processing sorting times are not an issue, sorting the complete volume will further speed-up the process

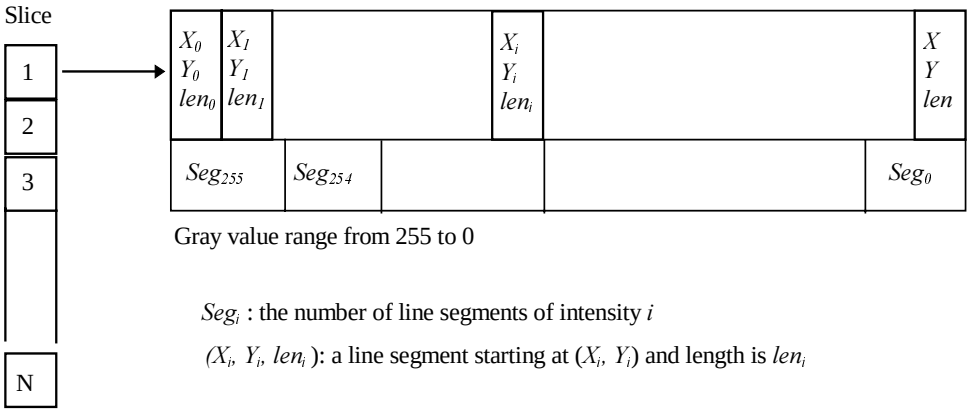


Figure 8. Line Encoding Data Structure

A last observation is that with real-life datasets it is obvious in MIP that the probability for a given grey value to be splatted in the final image is monotonically increasing with the value, i.e. is 100% for the value 255 and tends to 0% for the values close to 0. In addition, the contrast and "sharpness" of an image, what we intuitively perceive as "image quality", is determined almost exclusively by the quality of rendering of high values rather than of those close to zero. Based on this observation we decided to allow for chains in the lower values range a tolerance among an average value $v \pm \Delta v$ instead of requiring all values to be equal to v. The tolerance $\pm \Delta v$ depends on the value v, is 0 for v = 255 and gradually increases to a free adjustable value N for v = 0. In the current implementation, we employed a simple linear scheme with N typically having a value of 5~10. This results in longer chains for low values and, thus, further speed-up. Although this alters the image content, the effect is hardly visible due to the low probability of low values in the final image and their small contribution to the image contrast and sharpness! The encoding process becomes now:

```
Start_pos= pointer;  Start_intensity = *pointer;  len = 0;
while abs(*pointer++ - Start_intensity) < delta
        len++;
```

The *delta* is different for different *Start_intensity*, the higher the *Start_intensity*, the smaller the *delta* value (*delta* is range from 1 to N corresponding *Start_intensity* from 255 to 0).

5. Experiments

The experiments are performed on a SUN UltraSPARC 10 workstation based on the InViVo renderer [Saka93]. The goal of the experiments is twofold, to demonstrate the quality of our MIP splatting algorithm, and to prove the effectiveness of line encoded indexing, especially under interactivity requirement.

First, in order to compare the different quality modes, we designed a phantom with a resolution of 64×64×64 containing straight lines of one voxel thickness and rendered it with preview, standard and high quality mode respectively (see Figure 10). It is clear that in high quality mode images the line is smoother than the other two cases. In Figure 12, an MRA data set was rendered to compare the different modes including the delta-encoding effect. Four images were rendered by preview mode with delta-encoding, preview mode without delta-encoding, standard mode and high quality mode. Compared image *a* and image *b*, the difference in quality is very small, but the rendering time of image *a* is only 30% of that image *b*. This demonstrates the effectiveness of delta-encoding and sorting.

Second, in order to compare the quality difference among splatting shear-warp, projective shear-warp and ray casting, we also rendered the same phantom with these three methods and magnified it to show the details (see Figure 11). Ray casting image is typically sharper than other two images: due to the two interpolations, one during splatting and one during warping, shear-warp images (of any type) are always smoother.

Third, in order to test the speed of preview mode for MIP with encoding, we select a 3D ultrasound data set size of 256×256×256. Using the mip-map method we created also a compressed dataset with 128×128×128 resolution. The result image size is in both cases 300×300. We set the +Z axis as the viewing direction and calculated a complete rotation from 0° to 360° with an angle increment of 2° for both X and Y axes. We recorded the minimum, the maximum and the average time during rotation for all the following tests. Table 1, Table 2, and Table 3 list the time cost of ray casting MIP, splatting MIP and shear-warp MIP respectively (without run-length coding). In the case of the 256×256×256 dataset we notice that the average time cost of splatting is about two times more than the average cost of ray casting. This higher time cost is caused

by the fact that when splatting sheared (i.e. parallelepiped) voxels the projection area of each voxel is enlarged from 1.0 to maximum 4.0 (the average is about 2.25 if 0.5 is the average shear displacement along two axes). This means that one voxel will contribute to about 2.25 pixels in order to compare the maximum value. In shear-warp slices are sheared but voxels remain cubes, thus the projection area of each voxel is 1.0. This explains the faster performance of shear -warp as compared to splatting. Ray casting is still faster than shear-warp due to the fact that shear-warp provides always a gapless sampling accessing all voxels once. On the other hand, in this test case the image resolution of 300 is almost equal to the data resolution of 256 and ray casting usually does not sample all voxels (undersampling). The same tests have been done in the lower resolution data set of 128×128×128. In this situation the comparisons between shear-warp and splatting remain unchanged, however ray casting is now slower than both splatting and shear-warp due to the fact that using a ray resolution of 300×300 for sampling a dataset of 128×128×128 will cause each voxel to be sampled by several rays (oversampling).

Table 4 shows the results for preview mode with delta-encoding and sorting. Comparing with Table 2 showing the same method without any pre-processing, the speed-up achieved by our acceleration methods is 3.5 - 4.5! In Figure 13, we showed the images rendered by the three methods for 256 and 128 resolution. In Table 5, we listed memory cost for line encoding in this test.

Table 1 CPU times in seconds for ray casting preview quality (nearest neighbour, no interpolation)

Resolution/Time	Minimum	Maximum	Average
256	2.33	4.27	3.11
128	1.12	1.71	1.32

Table 2 CPU times in seconds for splatting preview quality (no interpolation, no run-length encoding)

Resolution/Time	Minimum	Maximum	Average
256	5.40	9.91	7.44
128	0.69	1.10	0.93

Table 3 CPU times in seconds for shear-warp preview quality

Resolution/Time	Minimum	Maximum	Average
256	3.77	4.48	4.02
128	0.35	0.49	0.40

Table 4 CPU time in seconds for splatting preview quality with run-length encoding and sorting

Resolution/Time	Minimum	Maximum	Average
256	1.17	2.37	1.70
128	0.17	0.37	0.26

Table 5 Memory Cost in MB for run-length line encoding

Resolution/memory	Original	Line encoded
256	16.0	6.245
128	2.00	1,017

6. Conclusions

In this paper, we presented the splatting MIP method in sheared object space and discussed the different footprints for different quality modes. According to our experiments, we conclude:

- Sheared object space is a suitable space to perform MIP in object space. In addition, shear-warp algorithms are ideal for maintaining an “optimal” sampling ratio, neither too intensive (oversampling) nor too sparse (undersampling).
- Splatting in sheared object space generates more accurate results than the original shear-warp.
- By choosing between different splatting and interpolating approaches (preview, standard, high quality), one can very effectively trade off quality for speed. Sub-sampling effects creating gaps are omitted in all cases.
- In object space volume rendering, we can design effective run-length encoding, including sorting methods, to accelerate the traversal process, minimise the number of memory accesses and optimise the cache performance.
- Nearest neighbour splatting with sorting provides the fastest way to preview volume data with interactive rendering speed even on low-range computers and therefore a reasonable way to interactive explore volumetric data.
- The benefit of splatting as compared to ray casting is accuracy and speed supported by voxel encoding and sorting.

References

[DrCH88] Drebin, R., Carpenter, L., and Hanrahan, P., *Volume Rendering*, ACM Computer Graphics (ACM SIGGRAPH'88 Proceedings), 22(4):65-74, August 1988

[HeMS95] Heidrich, W., McCool, M., and Stevens, J., *Interactive Maximum Projection Volume Rendering*, IEEE Proceedings Visualization'95:11-18, 1995

[IhLe95] Ihm, I., and Lee, R., *On Enhancing the Speed of Splatting with Indexing*, IEEE Proceedings Visualization'95:69-76, 1995

[Kaji84] Kajiya, T., *Ray Tracing Volume Densities*, ACM Computer Graphics (ACM SIGGRAPH'84 Proceedings), 18(3):165-174, July, 1984

[Knit95] Knittel, G., *High-speed Volume Rendering Using Redundant Block Compression*, IEEE Proceedings Visualization'95:176-183, 1995

[LaHa91] Laur, D., and Hanraham, P., *Hierarchical Splatting: A Progressive Refinement algorithm for Volume Rendering*, ACM Computer Graphics (ACM SIGGRAPH'91 Proceedings), 25(4)285-288, 1991

[LaLe94] Lacroute, P., and Levoy, M., *Fast Volume Rendering Using a Shear-warp Factorisation of the Viewing Transform*, ACM Computer Graphics (ACM SIGGRAPH'94 Proceedings), 28(3):451-459, July, 1994

[Levo88] Levoy, M., *Display of Surface from Volume Data*, IEEE CG&A, 8(3):29-37, May 1988

[LiGK97] Lippert, L., Gross, M., and Kurman, C., *Compression Domain Volume Rendering for Distributed Environments*, Computer Graphics Forum (EUROGRPHICS'97), 16(3):95-107, September, 1997

[Saka93] Sakas, G., *Interactive Volume Rendering of Large Fields*, The Visual Computer, 9(8):425-438, August 1993

[SaGS96] Sakas, G., Grimm, M., and Savopoulos, A., *Optimised Maximum Intensity Projection (MIP)*, technical report, 1996

[UpKe88] Upson, C., and Keeler, M., *V-BUFFER: Visible Volume Rendering*, ACM Computer Graphics (ACM SIGGRAPH'88 Proceedings), 22(4):59-64, August, 1988

[West90] Westover, L., *Footprint Evaluation for Volume Rendering*, ACM Computer Graphics (ACM SIGGRAPH'90 Proceedings), 24(2):367-376, 1990

[YeNR96] Yen, S., Napel, S., and Rubin, G., *Fast Sliding Thin Slab Volume Visualisation*, Proceedings of the 1996 Symposium on volume Visualization:79-86, San Francisco, October 1996

[ZuKV94] Zuiderveld, K., Koning, A., and Viergever, M., *Techniques for Speeding up High-quality Perspective Maximum Intensity Projection*, Pattern Recognition Letters, 15:507-517, 1994

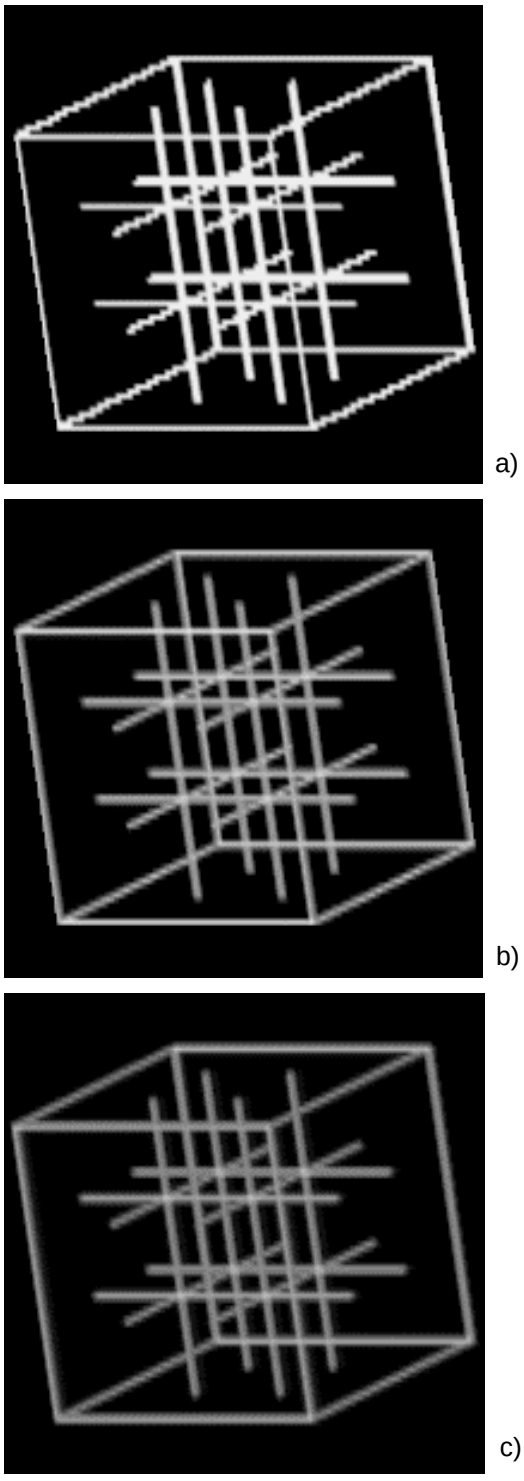


Figure 10. Rendering a phantom using splatting MIP with different quality a) Preview b) Standard c) High quality mode, no run-length encoding.

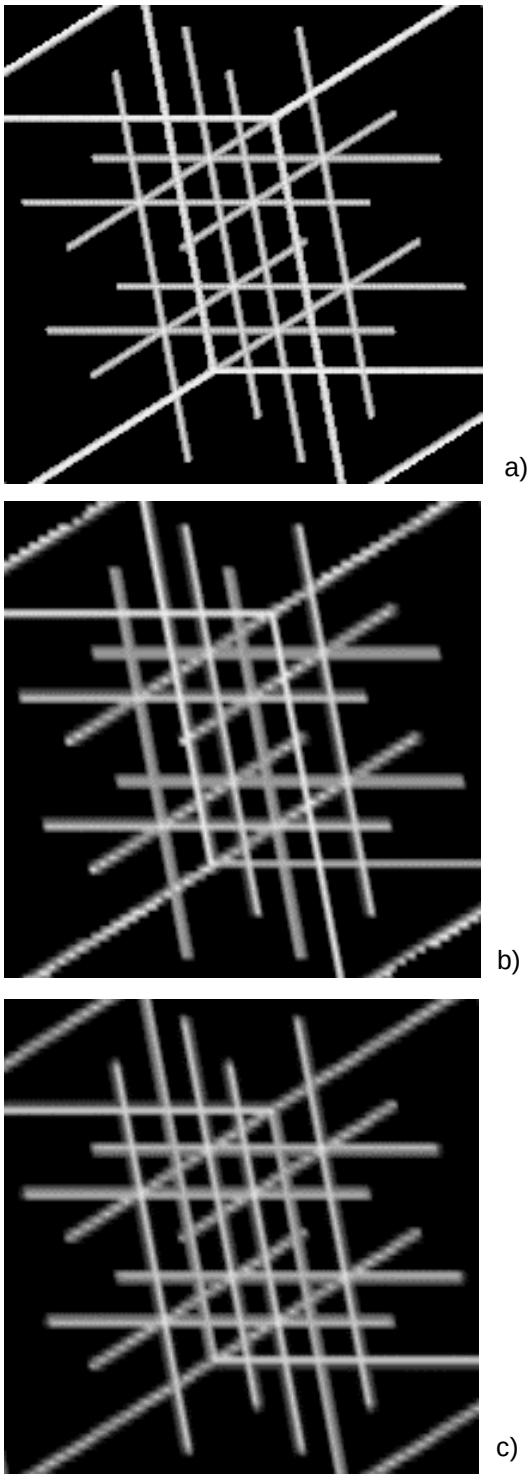
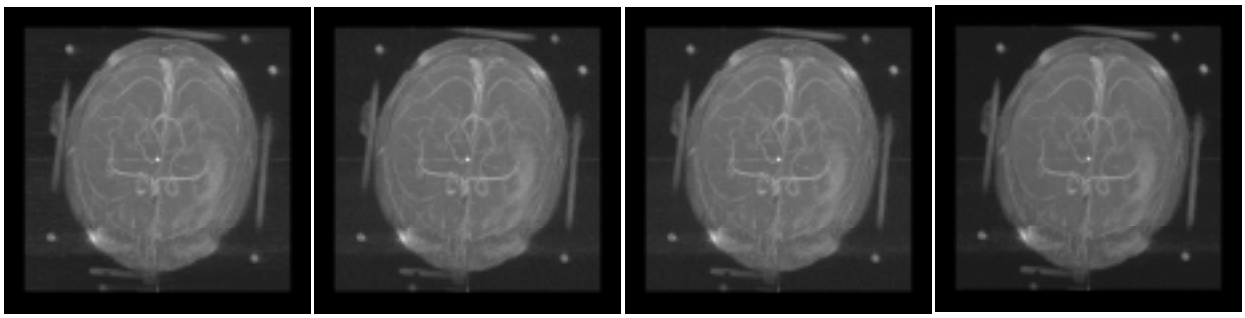
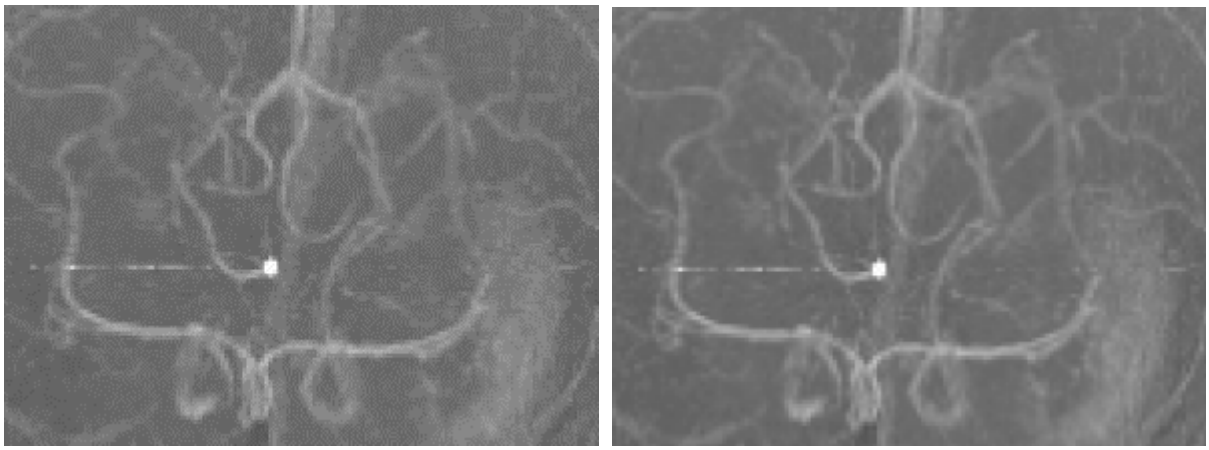


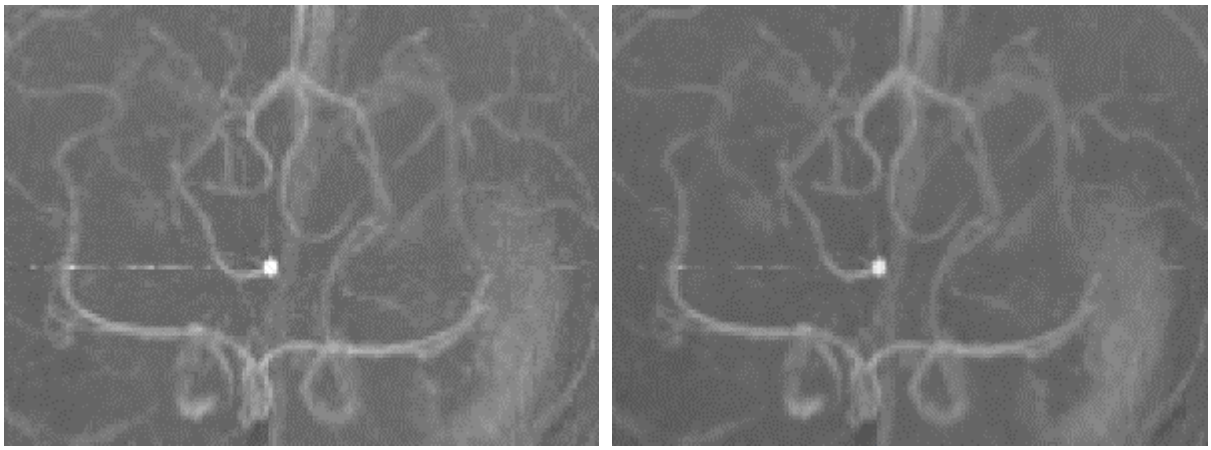
Figure 11. Rendering a phantom in “high” quality with a) ray-casting b) projective shear-warp c) splatting shear-warp



a) Overview b) Overview c) Overview d) Over-



a) Detail b) Detail



c) Detail d) Detail

Figure 12. Four rendering modes of MIP a) Preview with run-length encoding b) Preview without run-length encoding c) Standard mode d) High quality mode. Run-length encoding (a vs. b) has almost no influence on the image quality but effects in a speed-up of

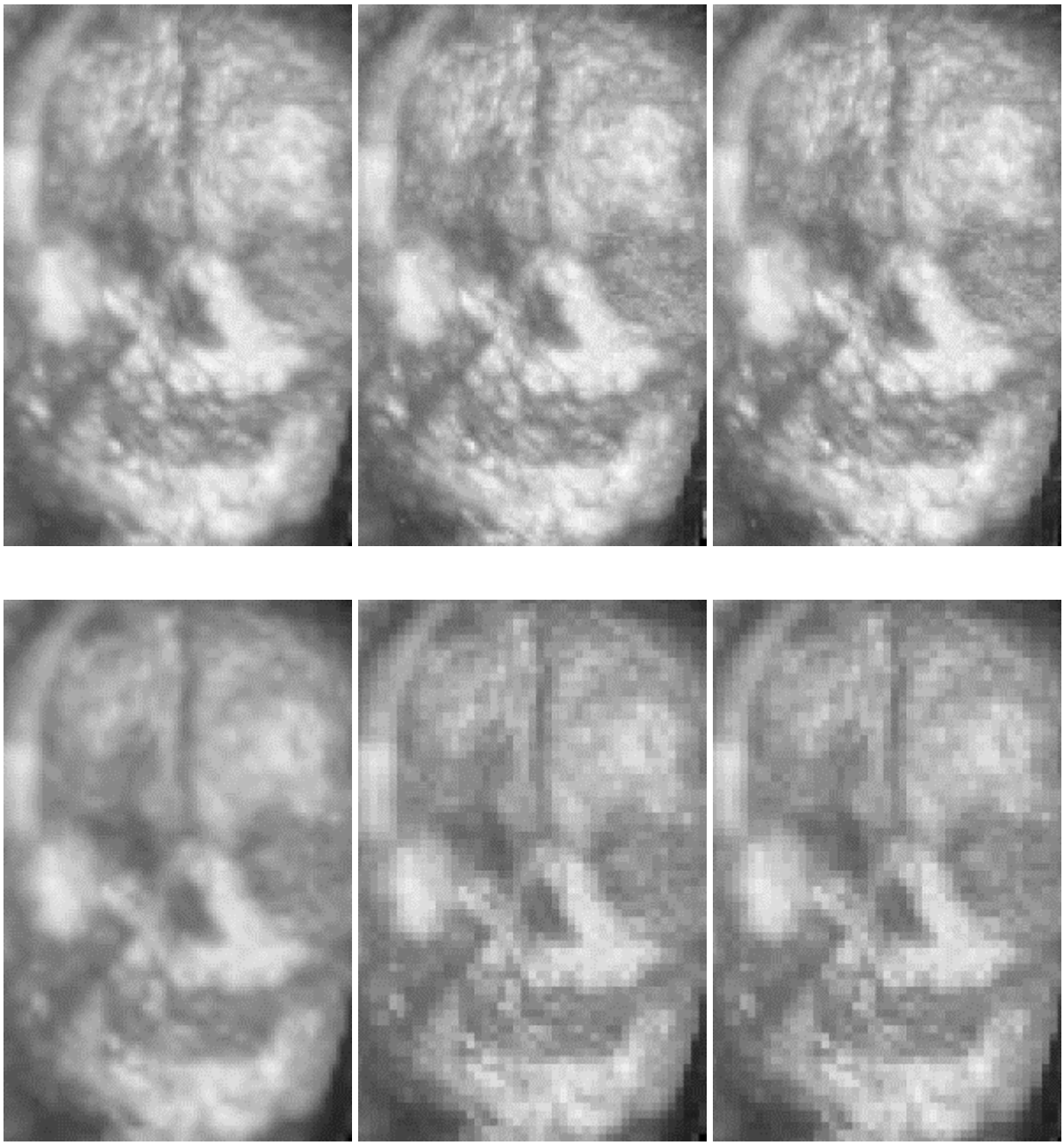


Figure 13. Comparing splatting, ray casting and shear-warp with 3D-ultrasound data sets of a fetus in two different resolutions. Splatting creates the smoothest images. The difference between ray casting and shear-warp is negligible.